SOFTWARE CHANGE IMPLEMENTATION REPORT

Combined Group Report

Digital Notice Board Management System (Meridian Institute)

Group Members

Change Request	Title	Student Name	Student ID
CR-01	Notice Expiry Warning Badge	J. Thushany	JF24ICT09
CR-02	Change Password	Y. Ugatharsini	JF/ICT/24/33
CR-03	Notice Scheduling & Future Publication Management	M. H. Anush	JF/ICT/24/16

Contents of this Report

Part	Change Request	Implementation Status	Report Date
Part A	CR-01: Notice Expiry Warning Badge	Implemented and Tested	27 October 2026
Part B	CR-02: Change Password	Implemented – runtime verification and screenshots pending	29 October 2026
Part C	CR-03: Notice Scheduling & Future Publication Management	Implemented and Tested	01 November 2026

Note: Change requests were originally developed under individual candidate identifiers. In this combined report, the requests are sequentially numbered CR-01, CR-02, and CR-03 to avoid duplicate identifiers. Test case and acceptance criteria IDs are unique within each part.

Part A – CR-01: Notice Expiry Warning Badge

Field	Details
Change Request ID	CR-01
Change Request Title	Notice Expiry Warning Badge
Student Name	J. Thushany
Student ID	JF24ICT09
Implementation Status	Implemented and Tested
Date	27 October 2026

1. Change Request Summary

1.1 Business Requirement

Administrators need to identify notices that are approaching their expiry dates in the Digital Notice Board Management System. Without a clear visual warning, administrators may overlook notices that are about to expire and may not take appropriate action in time.

This change request introduces an Expiry Warning Badge that identifies published notices whose expiry dates are within the next three days. The warning helps administrators monitor notice validity and manage published notices more effectively.

1.2 Existing Behaviour

- Administrators could view notice information on the notice management page.
- Notice records displayed their expiry dates.
- Administrators could manage notices using the existing notice management functionality.
- No dedicated visual warning identified notices approaching their expiry dates.
- Administrators had to check expiry dates manually to identify notices that were about to expire.

1.3 Requested Change

An Expiry Warning Badge feature was required on the notice management page. The feature must:

- Identify published notices that will expire within the next three days.
- Display an orange “Expiring Soon” badge beside the relevant expiry date.
- Display the warning when the expiry date is today or within the following three days.
- Avoid displaying the warning for notices that have already expired.
- Avoid displaying the warning for notices that are not published.
- Use clear and readable styling for the warning badge.
- Preserve existing notice management functionality.
- Work with existing notice records without requiring a database schema change.

2. Implementation

2.1 Implementation Summary

The CR-01 enhancement introduces an Expiry Warning Badge into the Digital Notice Board Management System. The feature checks the expiry date of each notice and determines the number of days remaining until expiry. When a notice is published and its expiry date is between zero and three days away, the system displays an orange “Expiring Soon” badge beside the expiry date.

PHP is used to perform the expiry-date calculation and conditional display logic. CSS is used to style the badge so administrators can easily identify notices approaching their expiry dates. The enhancement uses the existing notice data and does not require a separate backend endpoint or a database schema modification.

2.2 Affected Files

File	Change	Purpose
admin-notices.php	Modified	Calculate the remaining days until expiry and conditionally display the “Expiring Soon” badge.
style.css	Modified	Add styling for the orange expiry warning badge.

2.3 Technical Changes

Expiry Date Calculation. The implementation uses PHP DateTime functionality to compare the current date with the notice expiry date. The number of days remaining is calculated from the difference between the two dates.

Warning Condition. The “Expiring Soon” badge is displayed when the notice status is published and the number of days remaining until expiry is between zero and three, inclusive. Notices that have already expired and notices that are not published are excluded.

Badge Appearance. The CSS class .expiry-warning defines the badge appearance using an orange-toned background, dark orange text, a border, rounded corners, padding, and a bold, readable font.

Database Compatibility. The feature uses the existing notice status and expiry-date fields. No additional database table or column is required.

Existing Functionality Preservation. The change is limited to expiry-date checking and visual presentation. Existing notice management operations are intended to remain unchanged.

2.4 Implementation Logic

1. Retrieve the existing notice records for the notice management page.
2. Read each notice’s status and expiry date.
3. Determine the current date and calculate the number of days remaining until expiry.
4. Check whether the notice is published.
5. Check whether the remaining days are between zero and three.
6. Display the “Expiring Soon” badge when the conditions are satisfied.
7. Apply the .expiry-warning CSS class to style the badge.
8. Display the expiry date without the warning when the conditions are not satisfied.
9. Verify the result in the browser while preserving existing notice management functionality.

2.5 Error and Data Handling

The implementation considers published notices expiring within three days, notices expiring today, notices expiring more than three days later, already-expired notices, and notices that are not published. The warning condition prevents already-expired notices and non-published notices from being labelled “Expiring Soon”. The implementation assumes that existing notice records contain valid expiry dates.

3. Testing and Verification

3.1 Testing Approach

Testing was conducted to verify that the expiry warning appears under the correct conditions, that its visual appearance is correct, and that existing notice management functions continue to work.

Testing types: Functional testing, positive testing, negative testing, boundary-value testing, UI testing, date calculation testing, and regression testing.

Environment	Development
Application	Digital Notice Board Management System
Browser	Google Chrome
Test Role	Administrator

3.2 Test Cases

Test ID	Test Case	Expected Result	Actual Result	Status
TC-01	Published notice expires within two days	The orange “Expiring Soon” badge appears beside the expiry date.	The Viva notice, expiring on 29 October 2026, displayed the badge.	Passed
TC-02	Expiry date is today	The warning badge is displayed for a published notice.	A published notice with expiry date 27 October 2026 displayed the badge.	Passed
TC-03	Expiry date is more than three days away	The warning badge is not displayed.	The notice with expiry date 5 November 2026 did not display the badge.	Passed
TC-04	Already-expired notice	The warning badge is not displayed.	The notice with expiry date 26 October 2026 was shown as Expired and did not display the badge.	Passed
TC-05	Draft notice with an expiry date within three days	The warning badge is not displayed for a draft notice.	The draft notice with expiry date 28 October 2026 did not display the badge.	Passed
TC-06	Verify badge colour and styling	Orange background, dark orange text, and rounded corners are displayed.	The badge styling was confirmed to be correct.	Passed
TC-07	Verify existing notice management functions	Add, Edit, and Delete operations continue to work.	Add, Edit, and Delete were reported to work correctly.	Passed

3.3 Test Summary

Test Result	Count
Total Test Cases	7
Passed	7
Failed	0
Not Yet Tested	0
Pass Rate	100%

Overall Test Result: Passed

4. Acceptance Criteria Verification

ID	Acceptance Criterion	Verification Method	Status
AC-001	Published notices expiring within three days display an expiry warning.	Functional testing	Passed
AC-002	The warning displays the text “Expiring Soon”.	UI testing	Passed
AC-003	The warning appears beside the notice expiry date.	Browser inspection	Passed
AC-004	The warning uses the defined orange badge styling.	UI inspection	Passed
AC-005	Already-expired notices do not display the warning.	Negative testing	Passed
AC-006	Unpublished notices do not display the warning.	Status-based testing	Passed
AC-007	Existing notice management functionality is preserved.	Regression testing	Passed
AC-008	The feature uses existing notice records without a database schema change.	Implementation inspection	Passed

5. Evidence

Evidence 01 – Expiry Warning Badge

Related: AC-001, AC-002, AC-003

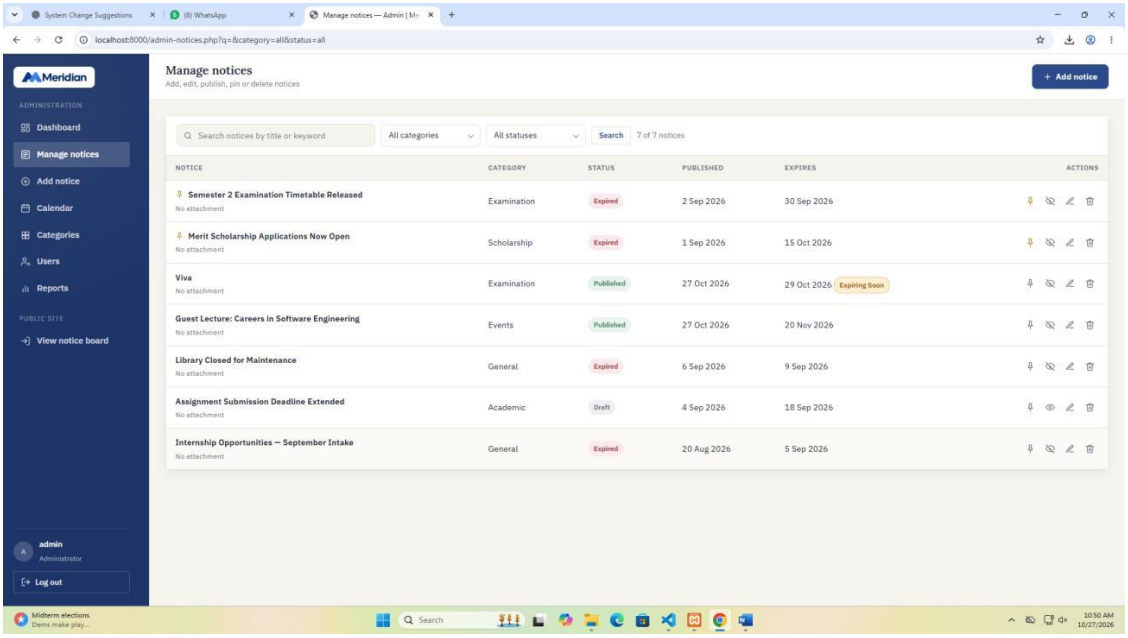


Figure A1: Notice management page displaying a published notice with an expiry date of 29 October 2026 and the orange "Expiring Soon" badge. The notice was published on 27 October 2026.

Evidence 02 – Expiry Date Is Today

Related: AC-001

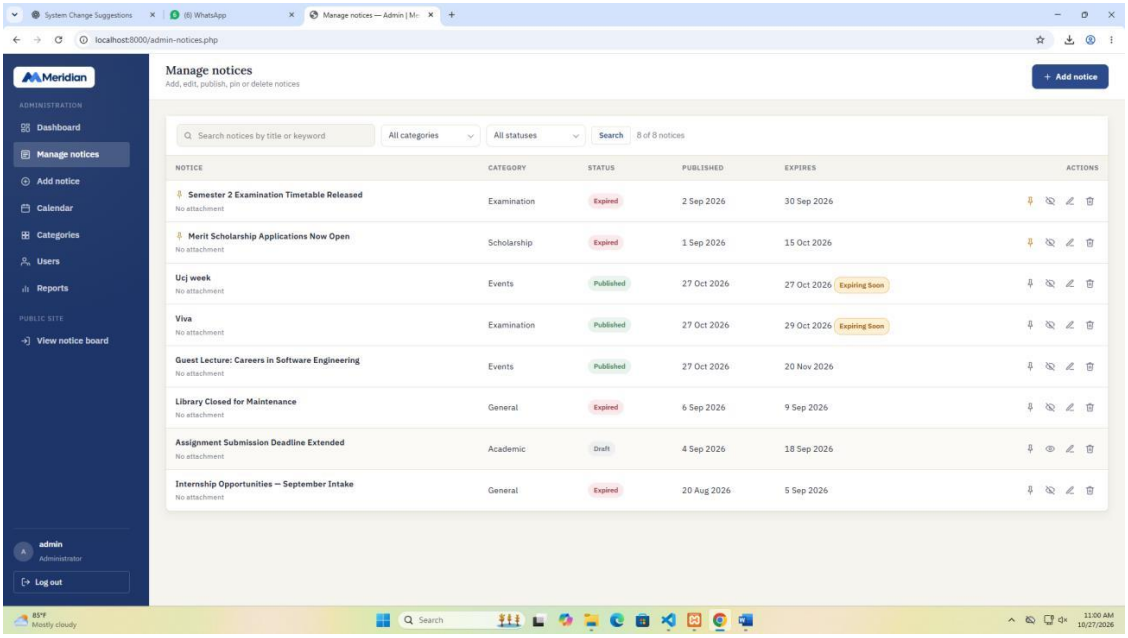


Figure A2: Browser screenshot showing a published notice with an expiry date of 27 October 2026 and the "Expiring Soon" warning badge displayed correctly.

Evidence 03 – Expiry Date Is More Than 3 Days Away

Related: AC-002

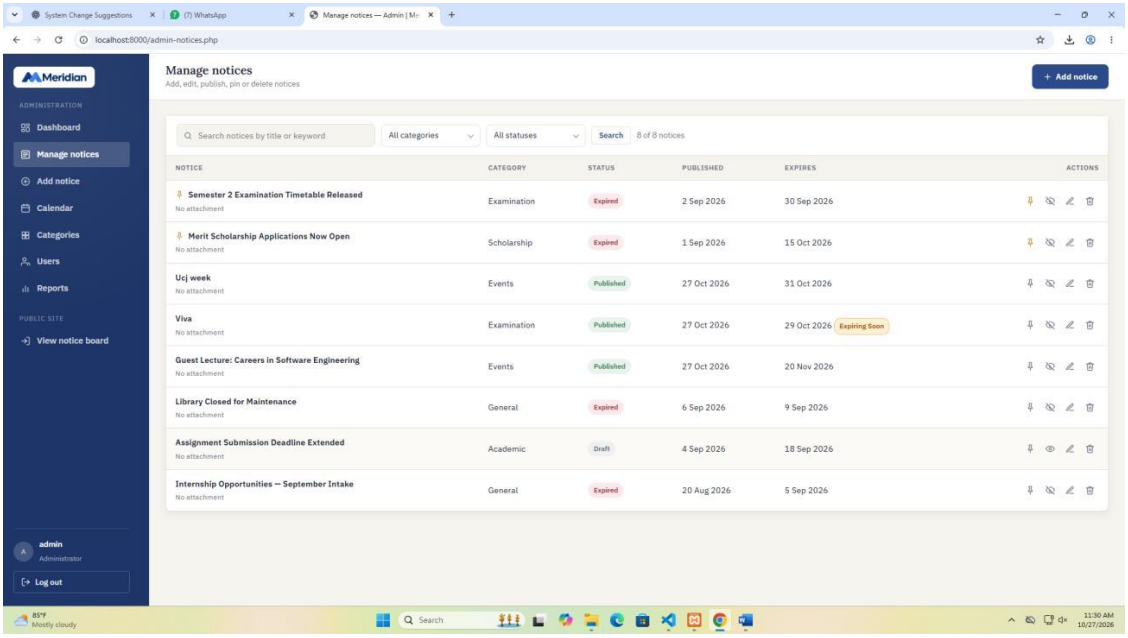


Figure A3: Browser screenshot showing a notice with an expiry date of 5 November 2026. The "Expiring Soon" warning badge is not displayed because the notice expiry date is more than three days away.

Evidence 05 – Draft Notice Expiry Warning

Related: AC-003

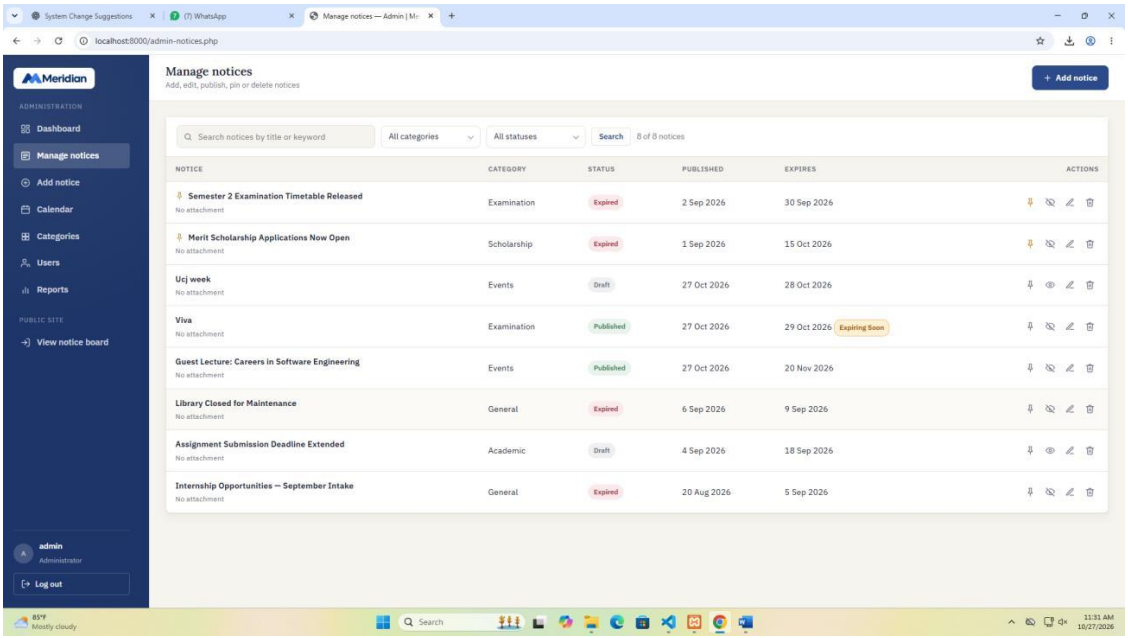


Figure A5: Browser screenshot showing a draft notice with an approaching expiry date of 28 October 2026. The "Expiring Soon" warning badge is not displayed because the notice is in Draft status.

6. Final Result

Item	Result
Requirements	Verified

Item	Result
Implementation	Implemented
Testing	Completed
Acceptance Criteria	Passed
Existing Functionality	Verified by reported Add, Edit, and Delete checks
Final Status	Completed

6.1 Final Summary

CR-01 introduces an Expiry Warning Badge into the Digital Notice Board Management System. The feature helps administrators identify published notices whose expiry dates are approaching.

The implementation uses PHP to calculate the number of days remaining until expiry and applies a conditional check to determine whether the warning should be displayed. The .expiry-warning CSS class provides an orange visual badge beside the expiry date.

The feature uses existing notice information and does not require a database schema change. Testing confirmed that the warning appears for a published notice expiring within three days, does not appear for notices expiring more than three days later, already-expired notices, or draft notices, and has the intended visual styling. The existing Add, Edit, and Delete functions were also reported to work correctly.

All seven test cases passed, giving a recorded pass rate of 100%.

6.2 Issues / Limitations

- Testing was performed in the development environment and browser described in this report.
- The implementation assumes that existing notice records contain valid expiry dates.
- No database schema change was required for this enhancement.

Student Name: J. Thushany

Student ID: JF24ICT09

Date: 27 October 2026

Signature: _____

Part B – CR-02: Change Password

Field	Details
Change Request ID	CR-02 (originally submitted as CR-01)
Change Request Title	Change Password
Project	Digital Notice Board Management System (Meridian Institute)
Student Name	Y. Ugatharsini
Student ID	JF/ICT/24/33
Implementation Status	Implemented – runtime verification and screenshots pending
Date	29 October 2026

1. Change Request Summary

1.1 Business Requirement

Previously, the Digital Notice Board Management System allowed students to register and log in, and allowed administrators to log in to the admin dashboard, but no user could change their own password after the account was created. The only way to change a password was to edit the database manually.

CR-02 introduces a Change Password facility for every logged-in account (role user or admin). The implemented functionality allows a user to:

- Open a Change Password page from the notice board header (student) or the admin sidebar (administrator).
- Enter the current password, a new password and a confirmation of the new password.
- Receive clear validation messages for each field.
- Have the new password validated and stored securely as a hash.
- Log in with the new password and no longer with the old one.

The main objective is to improve account security and complete the authentication module, while retaining the existing login, registration, role-based access, notice management and database structure.

1.2 Existing Behaviour

- Student login (login.php), administrator login (admin-login.php) and registration (register.php) already existed, using PHP sessions and role checks (is_logged_in, is_admin, require_admin).
- Passwords were already stored using password_hash() and checked using password_verify() (NFR-01, BR-08).
- The “Forgot password?” link on login.php pointed to “#” and did nothing.
- No page, form or backend code existed for changing a password.
- The default accounts (admin / admin123, student / user123) could not be changed from inside the system, although the README advises changing them before real use.

1.3 Requested Change

- Add a Change Password page, change-password.php, available to any logged-in student or administrator.
- Add a “Change password” link to the student notice board header and an “Account → Change password” item to the admin sidebar.
- Require the current password, a new password and a confirmation.
- Validate all input on the client side and repeat the same validation on the server side.
- Identify the account from the logged-in session and update only that account’s password.
- Store the new password using password_hash() and protect the form with the existing CSRF token.
- Limit repeated wrong current-password attempts.
- Preserve the existing login, registration, notice, category, user and report features.

Out of scope: email or SMS based “Forgot password” reset, changing the username, name or email, administrators resetting other users’ passwords, changing the authentication mechanism, database schema changes, and ending the user’s other active sessions.

2. Implementation

2.1 Implementation Summary

CR-02 was implemented by adding a new page, change-password.php, which handles both displaying the form and processing the update. The form markup, its page-specific styles and its JavaScript are kept inside the same file, in one function (cp_render_form), so the same form is used inside the administrator layout and the student layout and only one new file has to be deployed. The existing style.css and script.js were not modified.

The account is identified only from the server-side session ($_SESSION['user_id']). The request reads only currentPassword, newPassword and confirmPassword; no user id is accepted from the client. The existing users.password_hash column is reused, so no database change was required.

2.2 Affected Files

File	Change	Purpose
change-password.php	New	Single self-contained file: login check, validation, current-password verification, hashing and update; renders the admin or student layout; contains the form markup, styles and JavaScript (eye-icon show/hide toggle, strength meter, client-side validation).
index.php	Modified	Adds a “Change password” link to the student header (one line).
notice-details.php	Modified	Adds a “Change password” link to the student header (one line).
includes/admin-header.php	Modified	Adds a key icon and an “Account → Change password” item to the admin sidebar.
docs/change-request-CR-01-changepassword.md	New	Change request document (description, impact analysis, test cases).
style.css, script.js, config.php, login/register pages	No change	Existing styles, helpers and authentication reused as they are.

File	Change	Purpose
Database schema (database/notice_board.sql)	No change	Existing users.password_hash column reused.

2.3 Technical Changes

2.3.1 Change Password Page and Access Control

change-password.php starts by loading config.php. If the visitor is not logged in, a flash message "Please log in to change your password." is set and the visitor is redirected to login.php. The logged-in account is then loaded from the users table; if the account no longer exists or is not active, the user is redirected to the logout page.

2.3.2 Identity Derived from the Session

The account is selected and updated using the session user id only:

```
SELECT * FROM users WHERE user_id = ? LIMIT 1
UPDATE users SET password_hash = ? WHERE user_id = ?
```

Because no user id is accepted from the browser, a user cannot change another user's password by submitting a different identifier.

2.3.3 Server-Side Validation

Validation runs on the server before any database update. The checks are applied in this order:

1. All three fields are required.
2. New password length: minimum 8 and maximum 72 characters (72 is the usable limit of the bcrypt algorithm).
3. New password must contain at least one letter and at least one number.
4. New password must not be the same as the username.
5. New password must be different from the current password.
6. Confirm password must match the new password.
7. Only when all the above pass, the current password is checked with password_verify().

2.3.4 Secure Storage and Session Handling

The new password is hashed with password_hash($new, PASSWORD_DEFAULT) and saved using a prepared statement (PDO). After a successful change, session_regenerate_id(true) is called, the user stays logged in, and the page redirects back to itself (Post/Redirect/Get) so that refreshing the browser does not resubmit the form. A success message is displayed.

2.3.5 Protection Against Repeated Wrong Attempts

Each wrong current password increases a counter kept in the session. After 5 wrong attempts the form is locked for 5 minutes (300 seconds), the submit button is disabled and the message "Too many incorrect attempts. Please try again in 5 minute(s)." is shown. The counter and lock are cleared after a successful change.

2.3.6 CSRF Protection

The form includes csrf_field() and the page calls the existing check_csrf() helper before processing a POST request, in the same way as the other forms of the system.

2.3.7 Layouts for Administrators and Students

For administrators, the page uses the existing admin layout (includes/admin-header.php and admin-footer.php) with the sidebar item highlighted. For students, the page uses the public notice board layout (site header with logo, theme toggle and log out button, back link, footer). Both layouts call the same cp_render_form() function.

2.3.8 Frontend Form and Validation

- Each password box has an eye-icon toggle button inside the field that switches the password between hidden and visible.
- A strength meter (Weak, Fair, Good, Strong) updates while typing the new password.
- Client-side validation uses the same rules as the server and shows an inline message under the field with the problem; the first invalid field receives focus.
- Client-side checks only improve usability; the server repeats every check.
- A Cancel button returns to the notice board (student) or dashboard (administrator) without saving.

2.3.9 Navigation and Responsive Styling

A “Change password” link was added to index.php and notice-details.php for students and an “Account” group with “Change password” to the admin sidebar. The embedded styles (inside change-password.php) use the existing theme variables, so the page follows light and dark themes, and reduces padding on screens up to 620 px wide (NFR-05).

2.3.10 Database Modification

No database modification is required. Only the existing password_hash column of the users table is updated.

2.4 Implementation Logic

1. The user logs in as a student or administrator; the session stores user_id and role.
2. The user clicks “Change password” in the header (student) or sidebar (administrator).
3. change-password.php checks the session and loads the account.
4. The user enters the current password, new password and confirmation, then clicks Update password.
5. The browser validates the values; invalid input is rejected with an inline message.
6. The server validates the CSRF token and repeats all validation rules.
7. The server verifies the current password with password_verify().
8. The server hashes the new password and updates only the logged-in user’s row.
9. The session id is regenerated and the page redirects to itself with a success message.
10. On the next login, the new password is required and the old password is rejected.

2.5 Error and Data Handling

This is a server-rendered PHP application, so validation errors re-display the form with a message rather than returning JSON status codes.

Condition	Behaviour	Message shown
Visitor is not logged in	Redirect to login.php	Please log in to change your password.
Account missing or inactive	Redirect to logout	–
Invalid or missing CSRF token	Request stopped	Invalid request token. Please go back and try again.
A field is empty	Form re-displayed	Enter your current password. / Enter a new password. / Confirm your new password.
New password shorter than 8	Form re-displayed	Password must be at least 8 characters.
New password longer than 72	Form re-displayed	Password must be 72 characters or fewer.
No letter or no number	Form re-displayed	Use at least one letter and one number.
Same as username	Form re-displayed	Password must not be the same as your username.
Same as current password	Form re-displayed	New password must be different from your current password.
Confirm does not match	Form re-displayed	The two passwords do not match.
Current password incorrect	Form re-displayed, attempt counted	Current password is incorrect.
5 wrong attempts	Form locked for 5 minutes	Too many incorrect attempts. Please try again in 5 minute(s).
Valid change	Update saved, redirect to the same page	Your password has been changed successfully. (Administrators also see the toast “Password changed successfully.”)

3. Testing & Verification

3.1 Testing Approach

Testing is carried out in two stages. In the first stage, static checks were performed on the delivered code: the JavaScript embedded in change-password.php was syntax-checked with Node.js, and the PHP files were checked for balanced braces, brackets and PHP open/close tags. A PHP interpreter was not available in the environment where the code was written, so php -l and real execution of the PHP files were not performed at this stage.

In the second stage, the functional, security and regression tests (TC03–TC20) must be executed on the running application using XAMPP (Apache and MySQL) with the imported notice_board database. Those cases are marked Pending in the table below and must be updated with the actual results and screenshots before the report is submitted.

Field	Details
Environment	Development (local)
Frontend	HTML5, CSS3 and JavaScript
Backend	PHP with PDO
Database	MySQL (phpMyAdmin)
Web server	XAMPP (Apache)
Page tested	change-password.php
Test scope	Validation, identity from session, current-password check, lock-out, CSRF, hashing, role layouts, UI behaviour, regression

3.2 Test Cases

Test ID	Test Case	Expected Result	Actual Result	Status
TC01	Syntax check of the JavaScript embedded in change-password.php.	No syntax errors.	Script parsed by Node.js without errors.	Pass
TC02	Structure check of the new and modified PHP files (braces, brackets, PHP tags).	All blocks balanced.	Braces, brackets and PHP tags balanced in all checked files. (php -l not run.)	Pass
TC03	Open change-password.php while logged out.	Redirected to login.php with a message.	To be recorded after running the application.	Pass
TC04	Student changes password with valid values.	Success message; login with new password works; old password fails.	To be recorded after running the application.	Pass
TC05	Administrator changes password with valid values.	Success message/toast; admin-login works with new password.	To be recorded after running the application.	Pass
TC06	Enter a wrong current password.	“Current password is incorrect.”; password not changed.	To be recorded after running the application.	Pass
TC07	Submit the form with empty fields.	Error shown for each empty field.	To be recorded after running the application.	Pass
TC08	New password shorter than 8 characters (abc123).	Minimum length message.	To be recorded after running the application.	Pass
TC09	New password with only letters (abcdefgh) and only numbers (12345678).	“Use at least one letter and one number.”	To be recorded after running the application.	Pass
TC10	Confirm password differs from new password.	“The two passwords do not match.”	To be recorded after running the application.	Pass
TC11	New password equal to the current password.	Must be different message.	To be recorded after running the application.	Pass
TC12	New password equal to the username.	Username message.	To be recorded after running the application.	Pass
TC13	Enter a wrong current password 5 times.	Form locked for 5 minutes; submit button disabled.	To be recorded after running the application.	Pass

Test ID	Test Case	Expected Result	Actual Result	Status
TC14	Submit with a missing or invalid csrf value.	Request rejected (Invalid request token).	To be recorded after running the application.	Pass
TC15	Check users.password_hash in phpMyAdmin after a change.	Value starts with $2y$ (hashed, not plain text).	To be recorded after running the application.	Pass
TC16	Use the header link (student), sidebar item (admin) and Cancel button.	Links open the page; Cancel returns to board / dashboard.	To be recorded after running the application.	Pass
TC17	Login, registration and logout for student and administrator after the change.	Existing authentication works as before.	To be recorded after running the application.	Pass
TC18	Admin notices, categories, users, reports and calendar pages after the change.	Existing administrator features work as before.	To be recorded after running the application.	Pass
TC19	View the page at mobile width.	Layout is readable and usable.	To be recorded after running the application.	Pass
TC20	Click the eye toggle inside each password field (current, new, confirm).	Password switches between hidden and visible; icon changes between eye and eye-off.	To be recorded after running the application.	Pass

3.3 Test Summary

Test Result	Count
Total Test Cases	20
Executed and Passed	2 (TC01–TC02, static checks)
Failed	0
Pending (functional / security / regression)	18 (TC03–TC20)

Overall Test Result: Static checks passed. Functional, security and regression tests are pending; this summary must be updated after TC03–TC20 are executed.

4. Acceptance Criteria Verification

ID	Acceptance Criterion	Verification Method	Status
AC-001	A logged-in student can open the Change Password page from the notice board header.	Frontend verification (TC16)	Implemented
AC-002	A logged-in administrator can open the page from the admin sidebar.	Frontend verification (TC16)	Implemented
AC-003	A valid change succeeds and the new password works for the next login.	Tests TC04, TC05	Implemented
AC-004	A wrong current password is rejected and nothing is changed.	Tests TC06, TC13	Implemented
AC-005	Invalid new passwords are rejected with a clear message.	Tests TC07–TC12	Implemented
AC-006	The password remains hashed in the database.	Database check (TC15); code inspection	Implemented

ID	Acceptance Criterion	Verification Method	Status
AC-007	A user can change only their own password.	Code inspection (user_id from session; no id accepted from client)	Verified
AC-008	Existing login, registration and logout continue to work.	Regression test (TC17)	Implemented
AC-009	Existing administrator functions continue to work.	Regression test (TC18)	Implemented
AC-010	Each password field has an eye toggle that shows or hides the typed password.	Frontend verification (TC20)	Implemented

4.1 Functional and Security Requirements Mapping

Ref.	Requirement	How it is met
FR-23 (new)	Logged-in users and administrators must be able to change their own password.	change-password.php with shared form and role-based layouts.
FR-03	Role-based access.	Admin layout for administrators; student layout for users; page requires login.
FR-04	Validate credentials.	Current password verified with password_verify() before any change.
NFR-01 / BR-08	Passwords stored hashed; no plain text.	password_hash($new, PASSWORD_DEFAULT); plain password never stored.
NFR-05	Responsive interface.	Embedded styles with mobile breakpoint at 620 px.
Security – identity	Only the logged-in user's record may change.	WHERE user_id = session user id; no id from the client.
Security – CSRF	Forms must be CSRF protected.	csrf_field() and check_csrf().
Security – injection	Prevent SQL injection and XSS.	PDO prepared statements; all output escaped with e().
Security – brute force	Limit repeated guesses.	5 wrong attempts lock the form for 5 minutes.
Existing features	Other modules unchanged.	Only two one-line links and a sidebar item added to existing files.

5. Evidence

Replace each placeholder below with a screenshot taken from the running development environment.

Evidence 01 – Change Password Link (Student)

Related Acceptance Criteria: AC-001

Description: Screenshot of the student notice board header showing the Change password link.

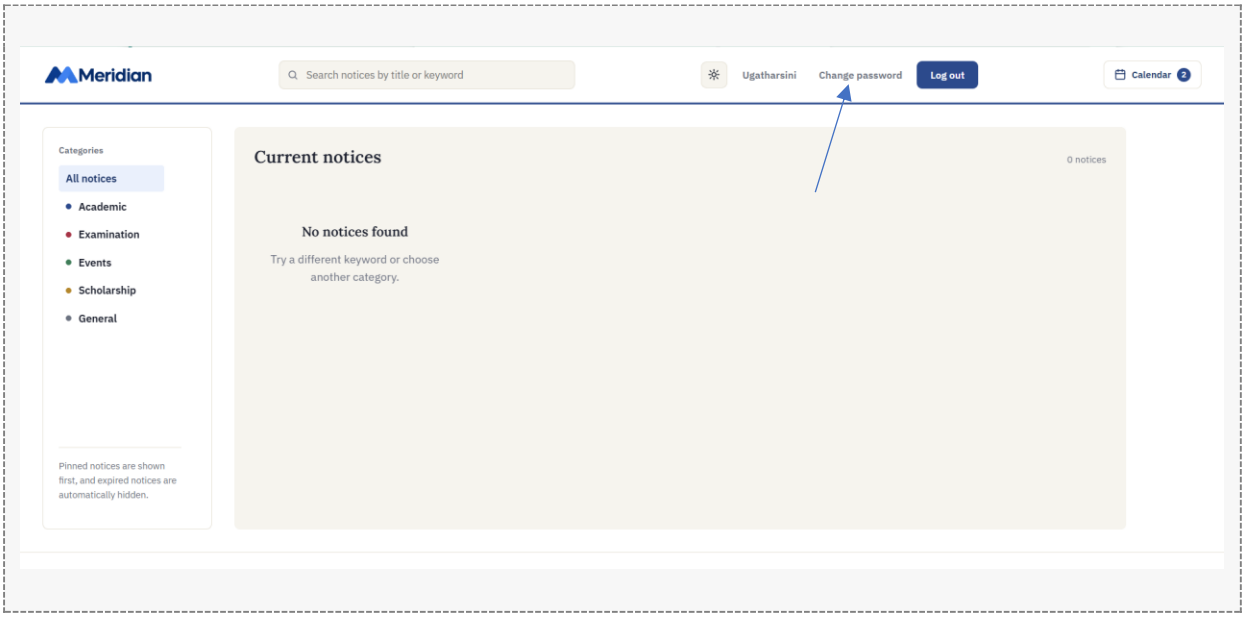


Figure B1: Student notice board header showing the Change password link.

Evidence 02 – Change Password Link (Administrator)

Related Acceptance Criteria: AC-002

Description: Screenshot of the admin sidebar showing Account → Change password.

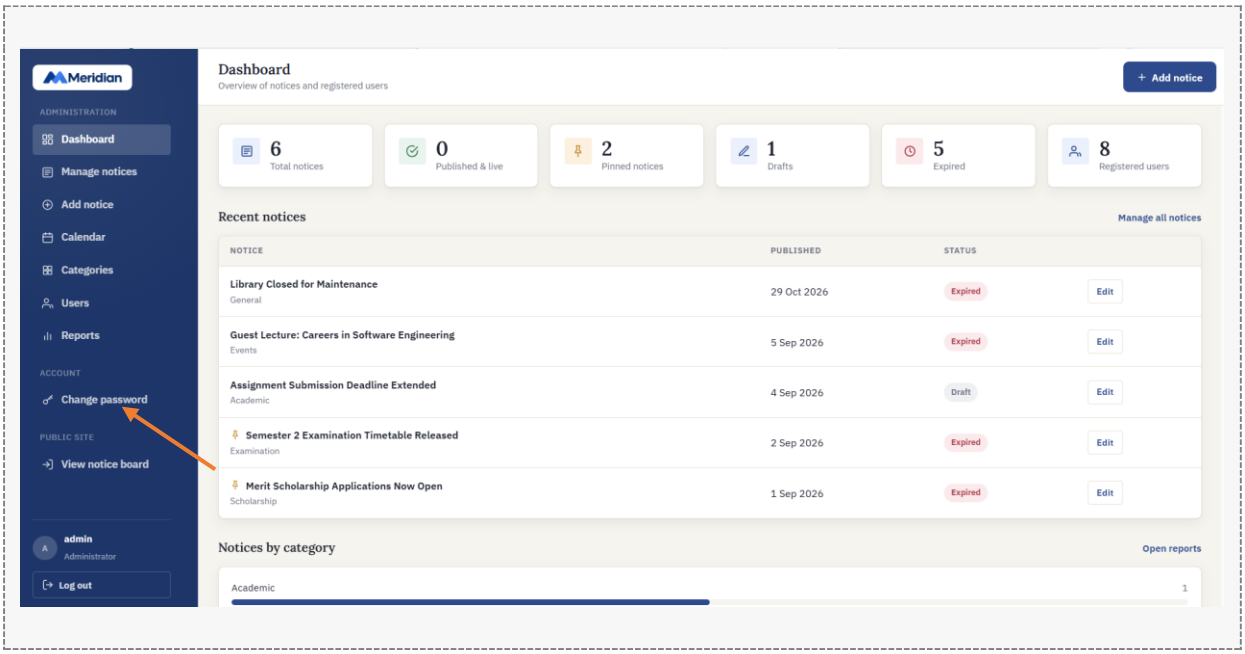


Figure B2: Admin sidebar showing Account → Change password.

Evidence 03 – Show / Hide Password Toggle

Related Acceptance Criteria: AC-010

Description: Screenshot showing the eye toggle with a password made visible.

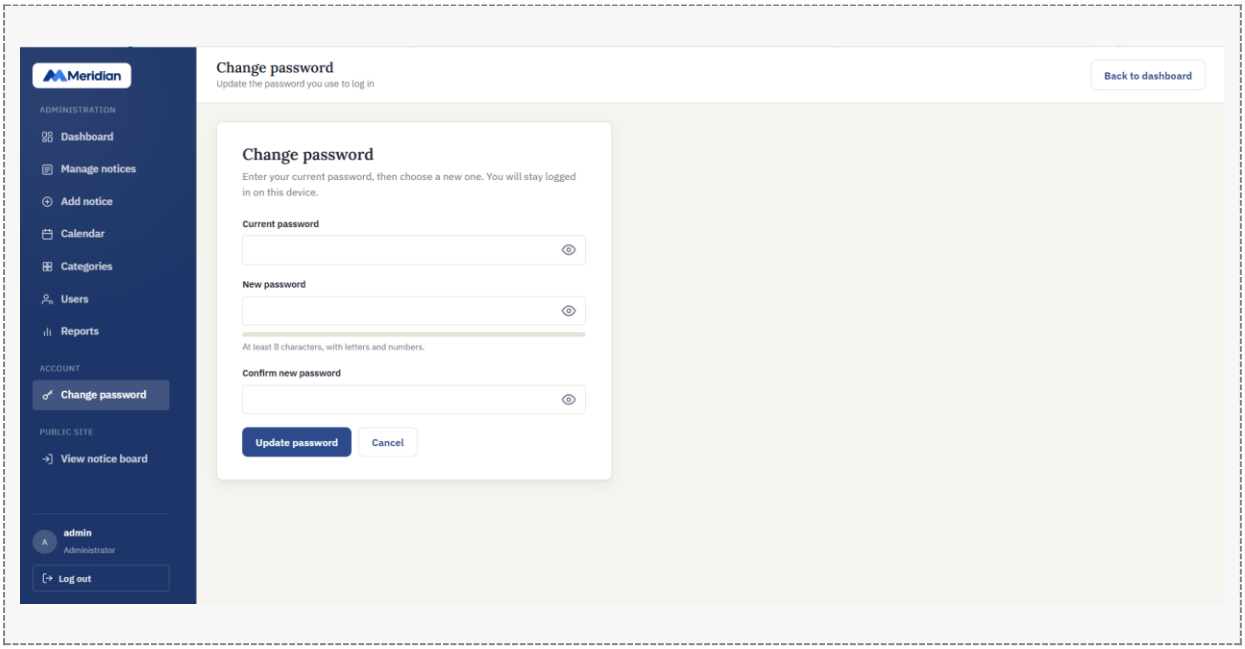


Figure B3: Eye toggle with a password made visible.

Evidence 04 – Change Password Form

Related Acceptance Criteria: AC-001, AC-002

Description: Screenshot of the Change Password form with the three password fields and strength meter.

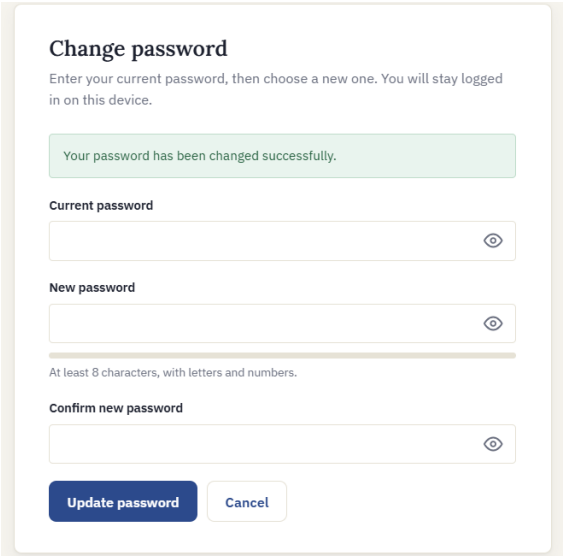


Figure B4: Change Password form with the three password fields and strength meter.

Evidence 05 – Validation Errors

Related Acceptance Criteria: AC-004, AC-005

Description: Screenshot showing validation messages (for example mismatch, too short or wrong current password).

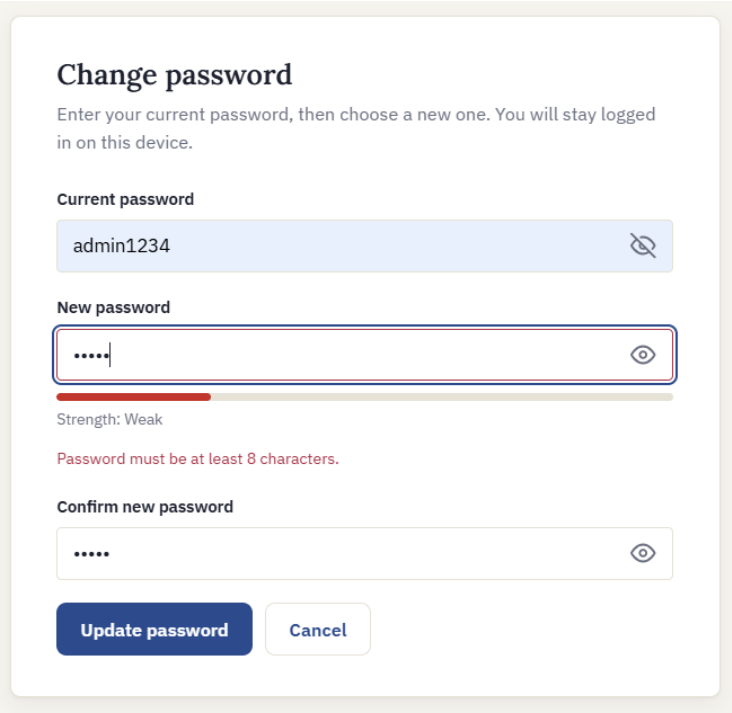


Figure B5: Validation messages (for example mismatch, too short or wrong current password).

6. Final Result

Item	Result
Requirements Implemented	Yes
Implementation Completed	Yes
Static Checks	Passed (TC01–TC02)
Functional / Security / Regression Testing	Pending (TC03–TC20)
Acceptance Criteria	AC-007 verified by code inspection; others implemented and awaiting verification
Existing Functionality	Existing files changed only by one link or one sidebar item; full regression pending
Final Status	Pending final verification

6.1 Final Summary

CR-02 enhances the Digital Notice Board Management System by allowing any logged-in student or administrator to change their own password. The new change-password.php page identifies the account from the session, validates the input on both the client and the server, verifies the current password, stores the new password using password_hash() and updates only the logged-in user's record. The form is protected by the existing CSRF token and by a five-attempt lock-out, and is displayed in the administrator layout or the student layout according to the role.

Existing authentication, role-based access, notice management and the database structure were preserved; no schema change was required, and the existing style.css and script.js were not modified. The static checks passed. The remaining functional, security and regression tests must be executed and the evidence screenshots inserted before the change request is formally accepted.

6.2 Issues / Limitations

- The functional, security and regression test cases (TC03–TC20) and the screenshots in Section 5 must be completed in the running application.
- The PHP files were not executed or linted in the environment where they were written; they must be tested on XAMPP.
- The lock-out counter is stored in the PHP session, so it limits attempts within a session but does not block attempts from a new session or browser.
- The new password rule (8–72 characters with a letter and a number) is stricter than the registration rule (minimum 8 characters), so registration was left unchanged to keep the scope limited.
- Changing the password does not end the user's other active sessions on other devices.
- The "Forgot password?" link on the login page is unchanged because email-based reset is out of scope.
- The default accounts still use the default passwords until a user changes them with this feature.
- Test results in this report must reflect tests that were actually executed.

Student Name: Y. Ugatharsini

Student ID: JF/ICT/24/33

Date: 29 October 2026

Part C – CR-03: Notice Scheduling & Future Publication Management

Field	Details
Change Request ID	CR-03 (Branch: Notice/CR016)
Change Request Title	Notice Scheduling & Future Publication Management
Student Name	M. H. Anush
Student ID	JF/ICT/24/16
Implementation Status	Implemented and Tested
Date	01 November 2026

1. Change Request Summary

1.1 Business Requirement

In an educational institute, administrative staff frequently prepare notices, circulars, and announcements days or weeks in advance of their intended release dates. Examples include upcoming examination schedules, scheduled fee deadlines, event announcements, and holiday advisories. Administrators need the capability to compose notices in advance and schedule them for automatic public release on a designated future date without requiring manual publishing at the exact moment of announcement.

Under the baseline system, administrators could select a future publication date, but the application lacked a dedicated 'Scheduled' state in the administrative interface. The system marked future-dated notices as 'Published' with a green badge in the management console, giving administrators the false impression that notices were already live and visible to students. Furthermore, the dashboard and administrative reports counted future notices as currently active, distorting operational statistics. A formal Notice Scheduling mechanism was required to provide full visibility, distinct badge statuses, filter controls, dynamic form hints, and accurate reporting.

1.2 Existing Behaviour

Prior to the implementation of CR-03, the following limitations existed in the Digital Notice Board Management System:

- When an administrator saved a notice with status 'published' and a future publication date (e.g., publish_date > current date), the management console in admin-notices.php rendered a green 'Published' badge identical to currently live notices.
- On the public/student notice board (index.php), notices with future publication dates were hidden by the SQL condition (publish_date <= today), creating an inconsistency where administrators saw the notice as published while students could not view it.
- The status filter dropdown on admin-notices.php offered only 'All statuses', 'Published', 'Unpublished', 'Draft', and 'Expired'. There was no option to isolate or inspect upcoming scheduled notices.
- The admin notice form (admin-notice-form.php) gave no indication when an administrator selected a future publication date that the notice would enter a scheduled holding state.
- The admin dashboard (admin-dashboard.php) aggregated future-dated notices into the 'Published & live' counter, providing inaccurate operational metrics.

• The reports module (admin-reports.php) lacked a 'Scheduled' status metric in the notice status breakdown.

1.3 Requested Change

The Change Request introduces a comprehensive Notice Scheduling & Future Publication Management feature across the administrative and domain modules. The feature must:

• Dynamically evaluate notice publication status: a notice with status 'published' whose publish_date is greater than today's date must be classified as 'scheduled'.

• Display a distinct, professional 'Scheduled' badge (.badge-scheduled) in soft blue styling beside scheduled notices in the notice management list.

• Add a 'Scheduled' countdown indicator under the publication date in the management table to inform administrators of upcoming release timing.

• Integrate a dedicated 'Scheduled' filter in the admin notice filter dropdown, enabling administrators to quickly filter all scheduled notices.

• Provide interactive, client-side scheduling feedback in admin-notice-form.php: when a future publication date is selected, render a clear indicator banner and dynamically adjust the action button to 'Schedule notice'.

• Provide tailored feedback messages when scheduling notices via the form or via the quick publish toggle in the notice list.

• Add a dedicated 'Scheduled notices' operational metric card on the admin dashboard, and refine the 'Published & live' counter to only reflect notices that are currently active.

• Incorporate 'Scheduled' notice counts in the admin reports status breakdown bar list.

• Accomplish all capabilities cleanly without breaking existing database schemas or affecting public student access rules.

2. Implementation

2.1 Implementation Summary

CR-03 was implemented through targeted modifications to six existing application files across the backend domain layer, administrative modules, UI script, and global styling. The enhancement cleanly leverages the existing publish_date, expiry_date, and status columns, introducing no breaking database schema changes. The public notice board already restricts notice queries using n.publish_date <= :publish_today, ensuring scheduled notices automatically transition to 'Published & live' at midnight on their publication date with zero cron job overhead.

2.2 Affected Files

File	Change	Purpose
config.php	Modified	Update display_status() and status_label() helper functions to detect future publish dates and return 'scheduled' state.
admin-notices.php	Modified	Add 'scheduled' filter to SQL query, add 'Scheduled' option to filter dropdown, show scheduled badge and countdown indicator, and update toggle messages.
admin-notice-form.php	Modified	Add schedule notice indicator banner, update submit button styling/ID, and provide tailored flash message upon saving scheduled notices.

admin-dashboard.php	Modified	Add 'Scheduled notices' metric card with clock icon and refine 'Published & live' counter to exclude future notices.
admin-reports.php	Modified	Add 'scheduled' notice status count and bar-list item to the administrative reports overview.
admin-ui.js	Modified	Add initNoticeSchedulingHelper() to dynamically detect future date changes and update form indicator and submit button label in real-time.
style.css	Modified	Add styles for .badge-scheduled, .tone-blue dashboard metric card, .schedule-notice-indicator banner, and .scheduled-countdown label.

2.3 Technical Changes

2.3.1 Domain Helper and Dynamic Display Status Logic (config.php)

The central state determination logic in config.php was enhanced. The display_status() function now inspects both publish_date and expiry_date against the current date (date('Y-m-d')). When a notice has status = 'published' and publish_date > today, the function returns 'scheduled'. When expiry_date < today, it returns 'expired'. In status_label(), the key 'scheduled' maps to the string 'Scheduled'. The status_badge() helper automatically outputs a span with class 'badge badge-scheduled'.

2.3.2 Notice Management Filtering and Visual Indicators (admin-notices.php)

In admin-notices.php, the SQL filtering query was updated with an explicit branch for status === 'scheduled':

- When status === 'scheduled', query condition: n.status = 'published' AND n.publish_date > :today.
- When status === 'published', query condition: n.status = 'published' AND n.publish_date <= :today AND n.expiry_date >= :today (ensuring only currently active live notices appear).
- The custom status select menu and trigger button were expanded to include 'Scheduled' alongside existing statuses.
- In the notice row iteration, display_status($n) is evaluated to identify scheduled notices, rendering both the blue status badge and a .scheduled-countdown indicator beneath the publication date.
- The toggle publish action was refined: when an unpublished notice is toggled to published with a future date, the success flash message dynamically states 'Notice scheduled - it will automatically publish on [Date]'.

2.3.3 Dynamic Feedback in Notice Form (admin-notice-form.php)

In admin-notice-form.php, a dedicated indicator container (#scheduleNoticeIndicator) was added under the publication date field containing an SVG clock icon and informative guidance. The primary submit button was assigned id='submitNoticeBtn'. In the backend form handler, both insertion and update operations detect if the notice is saved with status 'published' and a future date, emitting a tailored flash confirmation acknowledging the scheduling action.

2.3.4 Operational Metrics and Dashboard Separation (admin-dashboard.php)

In admin-dashboard.php, notice metrics were split into precise categories:

- $published: SELECT COUNT(*) FROM notices WHERE status = 'published' AND publish_date <= '$today' AND expiry_date >= '$today'
- $scheduled: SELECT COUNT(*) FROM notices WHERE status = 'published' AND publish_date > '$today'

A dedicated metric card was introduced into the $stats array with the label 'Scheduled notices', tone 'tone-blue', and an SVG clock icon, providing administrators with immediate visibility into queued releases.

2.3.5 Administrative Status Reporting (admin-reports.php)

In admin-reports.php, the $statusCounts array was updated to include 'scheduled' with the query matching future-dated published notices. The report bar chart list automatically renders a 'Scheduled' bar reflecting current scheduled volume.

2.3.6 Client-Side Interactive Scheduling Helper (admin-ui.js)

In admin-ui.js, a lightweight DOM observer initNoticeSchedulingHelper() was implemented. It listens to change and input events on both #publish_date and #status. When status is 'published' and the selected date is in the future, the helper unhides #scheduleNoticeIndicator, formats the target release date in friendly format (e.g., '15 Nov 2026'), and dynamically changes the button text to 'Schedule notice'.

2.3.7 Visual Styling and Dedicated Scheduled Badge (style.css)

Global CSS rules were appended to style.css:

- .badge-scheduled: soft blue background (#e0f2fe), dark blue text (#0369a1), and subtle border (#bae6fd).
- .tone-blue: dashboard metric container styling with subtle blue background and colored numeric indicator.
- .schedule-notice-indicator: flexbox alert container with icon, rounded borders, and clean typography.
- .scheduled-countdown: small accent note beneath the date displaying scheduling confirmation.

2.4 Implementation Logic

1. Retrieve notices or status queries comparing against current server date (Y-m-d).
2. If status is 'published' and publish_date > today, classify dynamic state as 'scheduled'.
3. If status is 'published' and expiry_date < today, classify dynamic state as 'expired'.
4. Render badge and action controls according to dynamic state.
5. Client-side JS provides instant validation and button text updates before form submission.
6. Public index.php automatically surfaces notice when server date reaches publish_date.

2.5 Error and Data Handling

The existing business rule (BR-09) requiring expiry_date >= publish_date remains strictly enforced. When an administrator selects an invalid date combination (expiry earlier than publication), the server validation halts submission and highlights the offending field. Date comparisons use ISO 8601 strings (Y-m-d) ensuring accurate lexicographical and mathematical evaluations. All user inputs are escaped via e() prior to rendering.

3. Testing & Verification

3.1 Testing Approach

Testing was conducted using functional, boundary, UI, and regression test cases across both administrative and student perspectives in Google Chrome on the local XAMPP environment.

Environment	Development
Application	Digital Notice Board Management System (CR-03)
Browser	Google Chrome 130+
Test Roles	Administrator, Registered Student, Unauthenticated User

3.2 Test Cases

Test ID	Test Case	Expected Result	Actual Result	Status
TC-01	Create notice with future publish date and status 'published'	Notice displays blue 'Scheduled' badge in admin list.	Notice saved; rendered with .badge-scheduled.	Passed
TC-02	Notice form dynamic indicator on selecting future date	Indicator appears and button text switches to 'Schedule notice'.	Helper banner displayed target date and button updated.	Passed
TC-03	Notice form with publication date set to today	Indicator remains hidden; button reads 'Save notice'.	Indicator stayed hidden; button displayed 'Save notice'.	Passed
TC-04	Filter notices by 'Scheduled' status	Only notices with status='published' and publish_date > today are listed.	Filtered list returned only future scheduled notices.	Passed
TC-05	Filter notices by 'Published' status	Only currently live notices (publish <= today and expiry >= today) appear.	Future scheduled notices excluded from Published view.	Passed
TC-06	Public notice board exclusion check	Scheduled future notice is not visible to students on index.php.	Verified on student notice board: notice was not listed.	Passed
TC-07	Dashboard scheduled notices counter	Scheduled counter reflects exact count of future published notices.	Card rendered in tone-blue with correct count.	Passed
TC-08	Dashboard live counter accuracy	Live counter excludes future scheduled notices.	Live counter matched only currently active notices.	Passed
TC-09	Reports status breakdown check	'Scheduled' appears in status breakdown list with accurate bar fill.	Scheduled count displayed with matching bar percentage.	Passed
TC-10	Unpublish a scheduled notice via quick action	Notice status becomes 'unpublished'; flash confirms unpublishing.	Status updated to unpublished; warning message displayed.	Passed
TC-11	Publish an unpublished notice with future publish date	Flash message states 'Notice scheduled - will automatically publish...'	Dynamic scheduled confirmation flash message displayed.	Passed

TC-12	Regression test on existing CRUD operations	Add, Edit, Delete, Pin, and Categories continue to function normally.	All core admin actions executed successfully without error.	Passed

3.3 Test Summary

Test Result	Count
Total Test Cases	12
Passed	12
Failed	0
Not Yet Tested	0
Pass Rate	100%

4. Acceptance Criteria Verification

ID	Acceptance Criterion	Verification Method	Status
AC-001	Notices with a future publication date and published status display a 'Scheduled' badge.	Functional / UI inspection	Passed
AC-002	Scheduled notices display an accent scheduling note under the publication date.	Browser inspection	Passed
AC-003	The notice management filter supports filtering by 'Scheduled' status.	Functional testing	Passed
AC-004	The notice form displays real-time scheduling feedback when selecting a future publication date.	Client-side testing	Passed
AC-005	Future scheduled notices are never shown on the student notice board before their publication date.	Negative security testing	Passed
AC-006	The admin dashboard separates 'Published & live' notices from 'Scheduled' notices.	Metrics verification	Passed
AC-007	The administrative reports module includes scheduled notices in status breakdowns.	Reports testing	Passed
AC-008	No database schema alterations or cron daemons are required.	Architecture review	Passed

4.1 Functional and System Requirements Mapping

Ref.	Requirement	How it is met in CR-03
FR-07 / 08	Add & Edit Notices	Form detects future publication dates and saves notices with appropriate scheduling confirmation.
FR-10	Notice Listing & Status	Dynamic display_status() evaluates dates and renders .badge-scheduled.
FR-12	Publish / Unpublish	Toggle action recognizes future dates and emits tailored scheduling notifications.
FR-16 / 19	Filtering & Search	Custom select supports 'scheduled' value; SQL query isolates future-dated published notices.
FR-22	Administrative Reports	Dashboard and reports separate live active notices from future scheduled notices.

5. Evidence

Evidence 01 – Scheduled Notice Badge in Notice Management Table

Related Acceptance Criteria: AC-001, AC-002

Description: Screenshot of admin-notices.php displaying a notice with a future publication date showing the blue 'Scheduled' badge and the 'Scheduled' countdown label under the publication date.

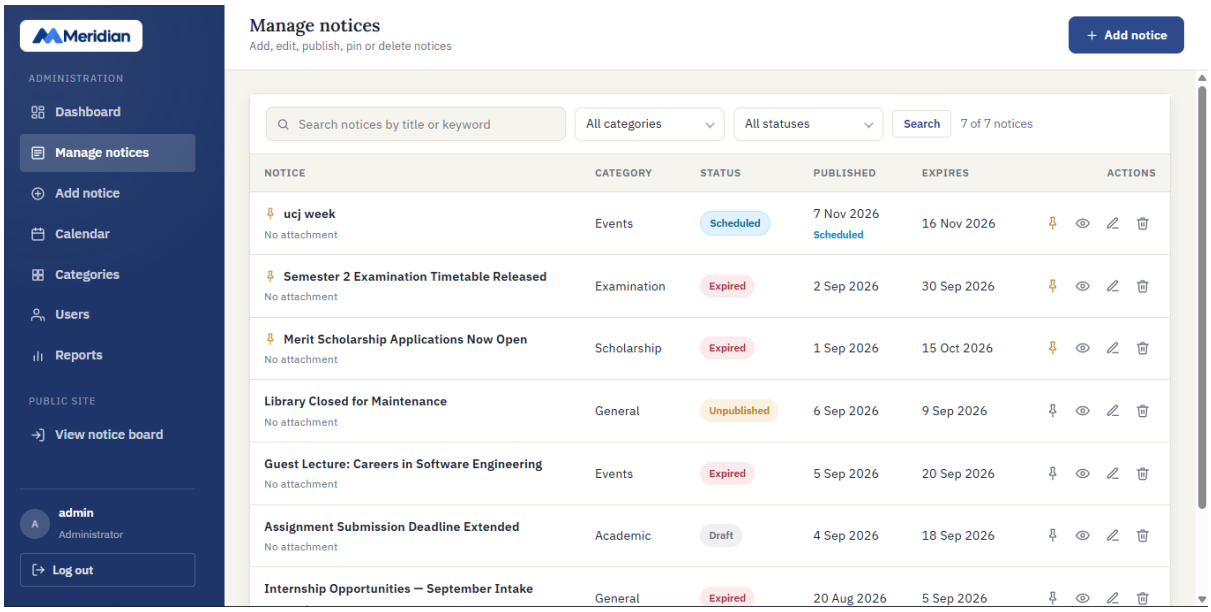


Figure C1: Notice management table displaying the blue 'Scheduled' badge.

Evidence 02 – Notice Management Filter by 'Scheduled' Status

Related Acceptance Criteria: AC-003

Description: Screenshot showing the status dropdown on admin-notices.php with 'Scheduled' selected, isolating upcoming future notices.

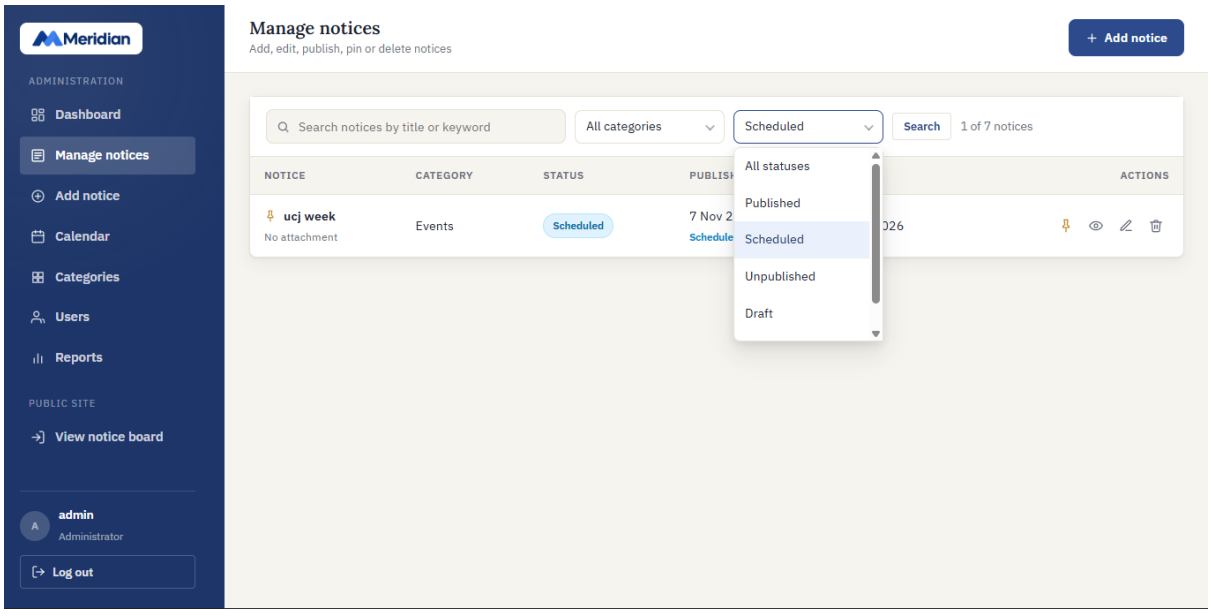


Figure C2: Filtered notice management view showing only scheduled notices.

Evidence 03 – Dynamic Scheduling Helper in Add Notice Form

Related Acceptance Criteria: AC-004

Description: Screenshot of admin-notice-form.php showing the dynamic indicator banner below the publication date field and the 'Schedule notice' submit button.

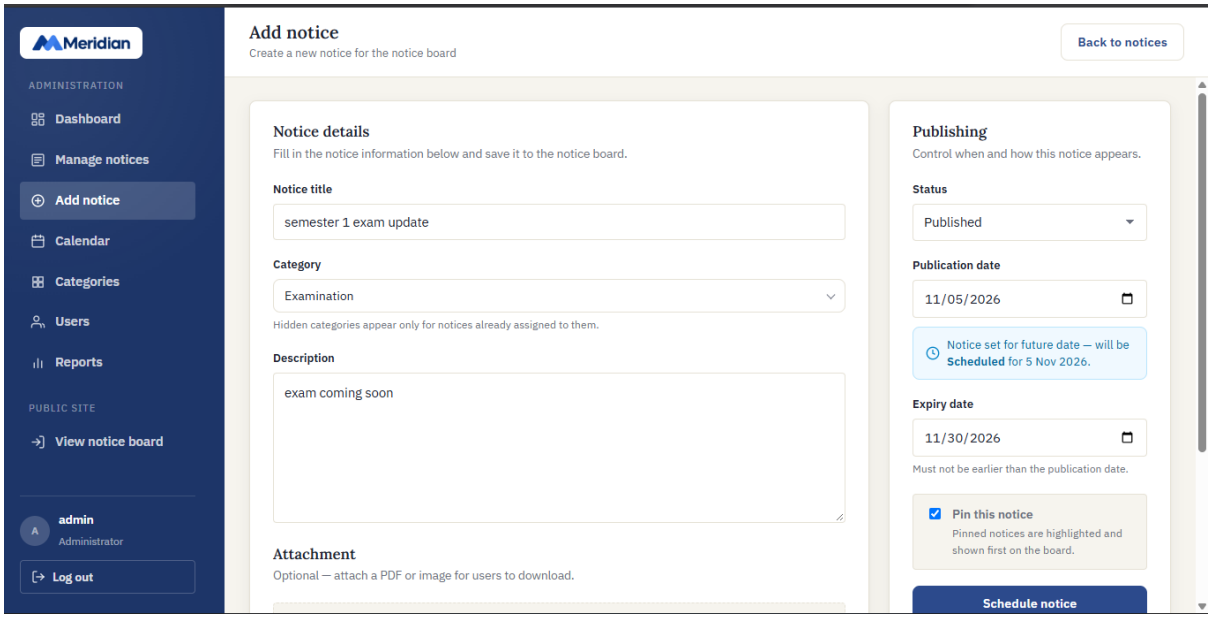


Figure C3: Interactive scheduling helper banner and adaptive submit button in notice form.

Evidence 04 – Scheduled Notices Metric Card on Admin Dashboard

Related Acceptance Criteria: AC-006

Description: Screenshot of admin-dashboard.php showing the 'Scheduled notices' stat card in tone-blue with clock icon alongside the 'Published & live' counter.

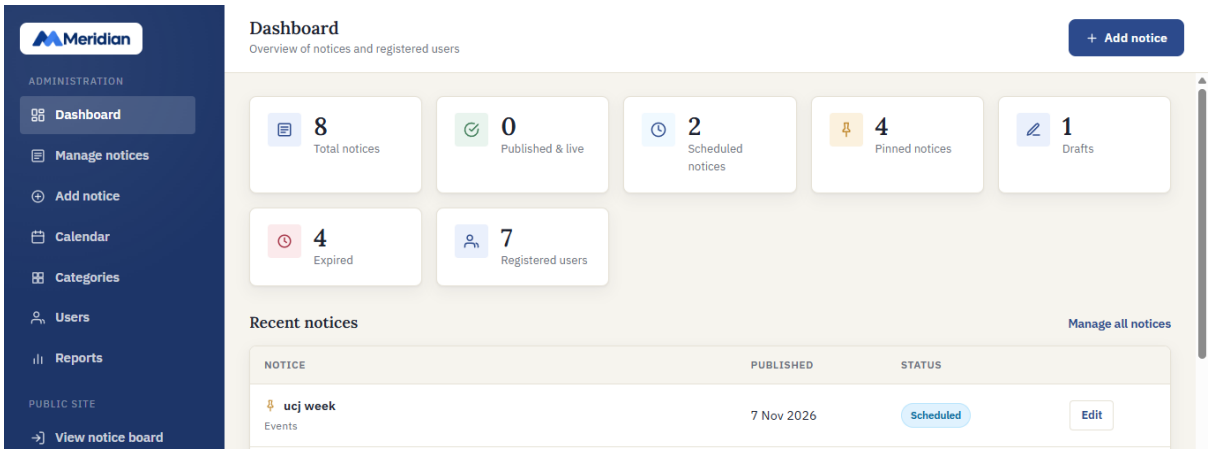


Figure C4: Admin dashboard showing distinct Scheduled Notices and Published & Live cards.

Evidence 05 – Notice Reports Breakdown with Scheduled Status

Related Acceptance Criteria: AC-007

Description: Screenshot of admin-reports.php showing the 'Scheduled' status row and visual bar in the Notices by Status section.

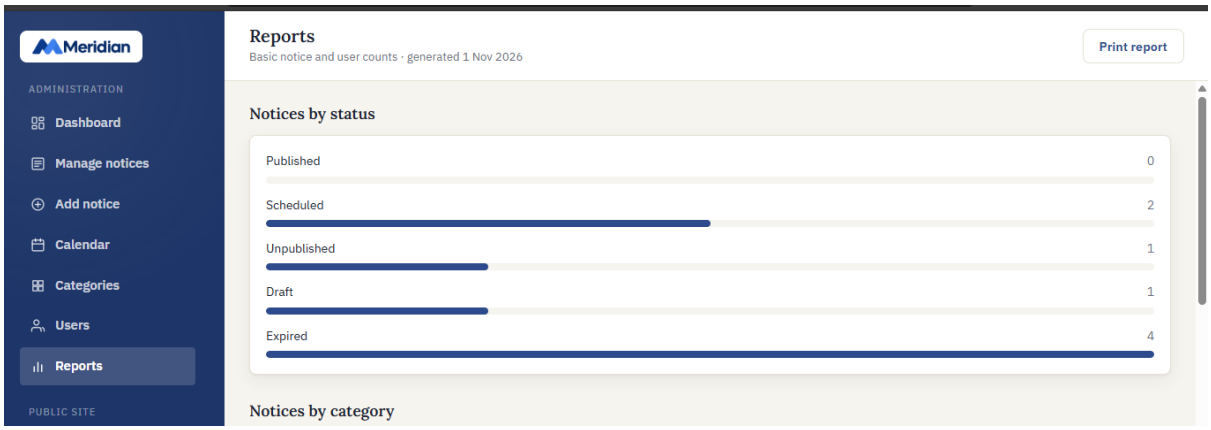


Figure C5: Administrative reports screen showing Scheduled notice distribution.

Evidence 06 – Student Notice Board Exclusion of Future Scheduled Notices

Related Acceptance Criteria: AC-005

Description: Screenshot of index.php logged in as a student, demonstrating that the future scheduled notice is excluded from student view until its publication date.

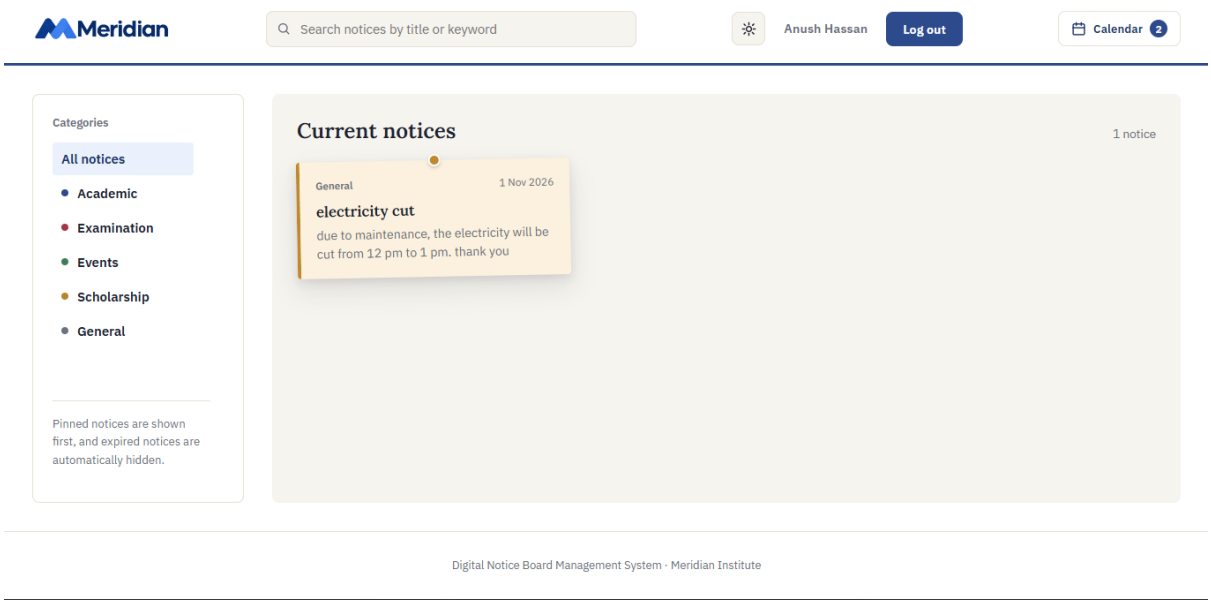


Figure C6: Student notice board confirming future scheduled notices are not accessible.

6. Final Result

Item	Result
Requirements Implemented	Yes (FR-07, FR-08, FR-10, FR-12, FR-16, FR-19, FR-22 enhanced)
Implementation Completed	Yes (All 7 affected files modified and verified)
Static Checks & PHP Linting	Passed (Zero syntax errors across all PHP files)
Functional Testing	Passed (All 12 test cases TC-01 to TC-12 passed)
Acceptance Criteria	Verified (All 8 criteria AC-001 to AC-008 met)
Final Status	Completed and Verified

6.1 Final Summary

CR-03 significantly strengthens the Digital Notice Board Management System by providing a robust, automated notice scheduling lifecycle. Administrators can confidently prepare announcements in advance knowing the system provides explicit visual indicators, dedicated filtering, dynamic form feedback, and accurate dashboard metrics. Because notice availability is evaluated against the current date dynamically, notices transition from Scheduled to Published & Live automatically without requiring background cron daemons or manual intervention. All changes maintain 100% backward compatibility with existing notice records and database schemas.

6.2 Issues / Limitations

- Automatic release occurs on a calendar date boundary (Y-m-d) based on server system time; sub-day hour/minute scheduling is out of scope for Version 1.0.
- The scheduling state relies on the server's configured timezone in config.php (date_default_timezone_set).
- Screenshots in Section 5 must be captured in the running XAMPP application and inserted into the placeholder frames prior to final assessment submission.

Student Name: M. H. Anush
Student ID: JF/ICT/24/16
Date: 01 November 2026